

CommonSource / Project_D ? Technical Overview

Purpose: Architecture reference, implementation status, and reusable context for future development (including AI-assisted work).

Workspace

1. Current Folder Structure

Project_D/

??? Project/

Notable absences: No embed.py in the repo (ingestion scripts import it). No package.json, Docker, or CI config in tree.

2. Frontend Pages & Components

No component framework (no React/Vue). Each page is a single HTML file with embedded CSS and vanilla JavaScript.

Page | Route | Role

landing.html

index.html functional areas

Area | Key JS

Articles tab |

3. Backend Modules

Module | Responsibility | Maturity

search_api.p

4. Current APIs

Base URL: http://localhost:5050

Static pages

GET /, /search, /join, /governance

Search & RAG

Method | Path | Description

GET | /api/s

Corpus & assets

GET /api/stats, /api/articles, /api/source/<id>, /api/models

Publisher / ingest (weak integration)

POST /api/publisher/register, /api/feed/add, /api/feeds/poll, /api/ingest/upload

GET /api/p

Caveat: ingest_rss helpers (poll_all_feeds, init_publisher_tables) may be incomplete; imports expect top-level

ingest_rss module.

5. Technologies Used

Layer | Stack

Backend | P

6. State Management

Frontend: Module-level let variables (currentQuery, lastSearchResults, swHistory, evidenceAbort). DOM is UI state. No Redux/localStorage.

Backend: SQLite durable state; in-memory embedding model + Ollama lock. No auth.

7. Existing Features

- ~3,116 articles, ~49,627 chunks (per /api/stats)

- Hybrid s

8. Missing Backend Functionality

- embed.py module (referenced, missing)

- Join form

9. Missing AI / RAG Functionality

- No vector ANN index (brute-force over capped SQL rows)

- No reran

10. Suggested Architecture Improvements

Short term: Restore embed.py; fix ingest imports; decouple slow layered load; gunicorn; wire join form.

Medium term: FTS5/sqlite-vec; background jobs for LLM; admin app.

Long term: Unify GAT world-models with search; modularize frontend; publisher dashboards + auth.

11. Current Project Flow

Search happy path: User ? index.html ? /api/search (embed + SQL + score) ? render cards ? /api/timeline ? /api/ask/layered (Qwen).

Corpus build: Archives ? metadata CSV ? ingest_commonsource ? commonsource.db ? optional extract_worldmodel.

Script Writer: /api/search (top 3) ? /api/generate per turn.

12. Frontend ? Backend Integration

API | index.html | Status

/api/search |

Dependencies: Flask on :5050; Ollama with qwen2.5:1.5b for Qwen features.

Implementation Status

Completed

SQLite corpus + search UI; layered Qwen; translation + Script Writer; static marketing pages; batch ingestion (historical).

Placeholders

join.html mock submit; publisher APIs incomplete; activateLayer() empty; GAT world-model tables unused online; search.py CLI broken.

Needs implementation

P0: embed.py + ingest paths. P1: join form + upload UI. P2: admin approval; world-model API; domain filters.

Reusable AI Context Block

Project: CommonSource (CMA) ? community media archive search + RAG demo.

Stack: Flask

System Design (Concise)

Style: Modular monolith ? Flask owns HTTP, retrieval, ranking, Ollama.

Data model:

knowledge_assets ??< knowledge_chunks (embedding_blob)

???

Evidence layers: news, development, community, official ? via source_classifier.

Deployment today: Local Flask + Ollama + browser on localhost:5050.

Generated architecture overview for Project_D / CommonSource.